SDPD / WEEKS 1–6

Java Foundations

Student notes

From your first Java program to decisions, loops, text and random numbers.

Read the explanation.
Predict the result.
Run the code and explore.

Software Design and Programme Development
Atlantic Technological University

Course material updated 19 October 2026
PDF edition · November 2026

YOUR GUIDE TO THE NOTES

Contents

Eight chapters, from the module overview to a six-week review.

01 **Module guide** 3
Purpose, learning outcomes and assessment

02 **Week 1 · Java foundations** 6
Programs, output, variables, types and arithmetic

03 **Week 2 · Output and calculations** 19
Writing Java programs and working with values

04 **Week 3 · Input and decisions** 29
Scanner input, comparisons and if statements

05 **Week 4 · More decisions** 37
Logical operators, strings and switch

06 **Week 5 · Loops and formatting** 44
Repetition, validation and formatted output

07 **Week 6 · Types, text and chance** 53
Casting, calculations, characters, strings and random numbers

08 **Review · Weeks 1–6** 69
A guide to revisiting the main ideas

How to use these notes
Start with the complete programs. Examples labelled **Inside main** belong inside your main method. Predict each result, run it in IntelliJ IDEA, then change a value and try again. Use the contents links or PDF bookmarks to find a chapter.

CHAPTER 01

Module guide

Purpose, learning outcomes and assessment

What this module is about

Software design transforms **user requirements** into a form that can guide coding and implementation. Program development then carries an idea through specification, design, programming, documentation, testing, debugging and maintenance.

The practical work is where these ideas become usable programming skills. The overall goal is to turn a need into a tested, maintainable program.



Software design and program development

Weekly structure

There are three student groups: A, B and C. The lectures are common to all three groups, while each group should use its own timetable for group-specific classes.

Day	Common lecture
Monday	11:00
Thursday	13:00
Thursday	16:00

Each teaching week contains three one-hour lectures and three practical lab sessions.

Activity	Weekly allocation
Three one-hour lectures	3 hours
Three practical lab sessions	6 hours
Total class time	**9 hours**

Learning outcomes

By the end of the module, students should be able to:

- understand the basic concepts behind key programming structures;
- select appropriate programming constructs for a problem;
- implement those constructs to solve a variety of problems;
- understand a software specification and prepare suitable test data;
- demonstrate competence in the fundamentals of software development; and
- work effectively both individually and as a member of a team.

Assessment

Assessment is distributed across both semesters. The timings below are shown by stage rather than by fixed calendar date.

Semester	Stage	Assessments	Weight
Semester 1	Mid-semester	Practical lab exam 1 (10%) MCQ 1 (7.5%)	**17.5%**
	End of semester	Practical lab exam 2 (10%) MCQ 2 (7.5%)	**17.5%**
Semester 2	Throughout	Weekly lab micro-assessments (10 × 1%)	**10%**
	Mid-semester	Practical lab exam 3 (20%) MCQ 3 (7.5%)	**27.5%**
	End of semester	Practical lab exam 4 (20%) MCQ 4 (7.5%)	**27.5%**
After Semester 2	Early May	Optional MCQ 5	—

The practical lab exams contribute 60%, weekly lab micro-assessments contribute 10%, and MCQs contribute 30%. MCQ 5 is optional: the best four results out of five count, so it can replace a missed attempt or improve a score.

The first topics

Topic	Why it matters
Program structure	Recognise classes, methods, blocks and statements.
Text output	Display messages with `print` and `println`.
Variables and types	Store values and describe what they represent.
Operators	Calculate, compare and combine values.
Input and conversion	Let a program receive and use data.

CHAPTER 02

Week 1 · Java foundations

Programs, output, variables, types and arithmetic

What is programming?

A **program** is a sequence of instructions that specifies how a computer performs a computation. That computation may be mathematical, data-driven or one part of a larger software system.

Most programming languages provide constructs for:

- **input** — receiving data from a keyboard, file, sensor or another device;
- **output** — displaying or sending data;
- **calculation** — performing mathematical operations;
- **decision** — choosing an action based on a condition; and
- **repetition** — performing an action repeatedly.

A short history of programming

Year	Development	Why it matters
1843	Ada Lovelace described an algorithm for Charles Babbage's Analytical Engine.	Her Bernoulli-number algorithm is often recognised as the first published computer program.
1957	John Backus's team at IBM developed FORTRAN.	**FOR**mula **TRAN**slation made high-level scientific programming practical for a wider audience.
1959	COBOL was designed for business data processing.	It became important in large organisations; Grace Hopper was among the influential early contributors.
1972	Dennis Ritchie developed C at Bell Labs.	C became closely associated with Unix and influenced many later languages.
1979	Bjarne Stroustrup began developing C with Classes.	It became C++, adding object-oriented features while retaining much of C's syntax.
1995	James Gosling and the Sun Microsystems team released Java.	Java uses a virtual machine so the same bytecode can run on different platforms.

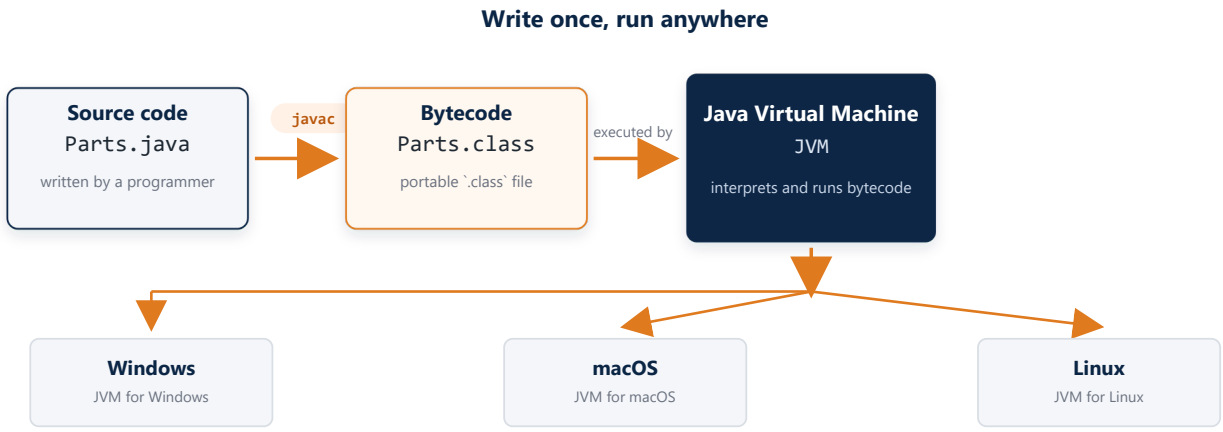
Why Java?

Java is used in this module for three main reasons:

1. **Platform independence.** Java source is compiled to bytecode, which a compatible Java Virtual Machine can run on Windows, macOS, Linux and other platforms.
2. **Clear structure.** Java has a consistent structure for classes, methods, data and behaviour, providing a clear route from a problem description to a working program.
3. **Robust software.** Strong type checking, automatic memory management and exception handling help reveal errors and support maintainable programs.

How Java works

Source code is written in a `.java` file. The compiler checks it and produces a `.class` file containing **bytecode**. Bytecode targets the Java Virtual Machine (JVM), rather than one particular processor, so a platform-specific JVM can execute the same compiled file on different operating systems.



Java source code is compiled to portable bytecode and executed by a platform-specific JVM

This is the idea behind **write once, run anywhere**: compile the source once, then let the JVM for each platform handle execution.

IntelliJ IDEA

IntelliJ IDEA is an **integrated development environment (IDE)**. It provides a code editor, project management, compiler integration, run controls and debugging tools. The course examples use IntelliJ IDEA to edit, compile and run Java programs.



The IntelliJ IDEA editor and Run window

The complete program

```java
// A simple Java program

public class Parts {
    public static void main(String[] args) {
        System.out.println("I love programming!");
    }
}
```

CONSOLE OUTPUT

```
I love programming!
```

The program has one class named `Parts`. When the program runs, execution begins in the `main` method and the statement inside it displays a message.

Parts of a Java program

Part	Meaning
`public`	An access modifier. Here it makes the class available wherever normal Java access rules permit. Java keywords are lowercase.
`class`	Tells the compiler that a class definition is beginning.
`Parts`	The programmer-chosen class name. It is an identifier.
`{ ... }` after the class name	Defines the class body: the container for fields, constructors and methods belonging to the class.
`public static void main(String[] args)`	The header of the `main` method, which is the starting point for this introductory standalone application.
`{ ... }` after `main`	Defines the method body. Statements inside it run when `main` is called.
`System.out.println(...)`	Calls Java's standard output method to display text and then move to a new line.
`;`	Marks the end of this Java statement.

Reading the structure in code

Because `Parts` is declared `public`, the source file must be named `Parts.java`. The text inside quotation marks is printed without the quotation marks.

Not every line ends with a semicolon. Class and method headers, braces, comments and some control structures follow different rules. Blank lines also do not change what the program does, but they make its structure easier to see.

```
public class Parts {

    public static void main(String[] args) {
        System.out.println("I love programming!");
    }
}
```

Compile and run

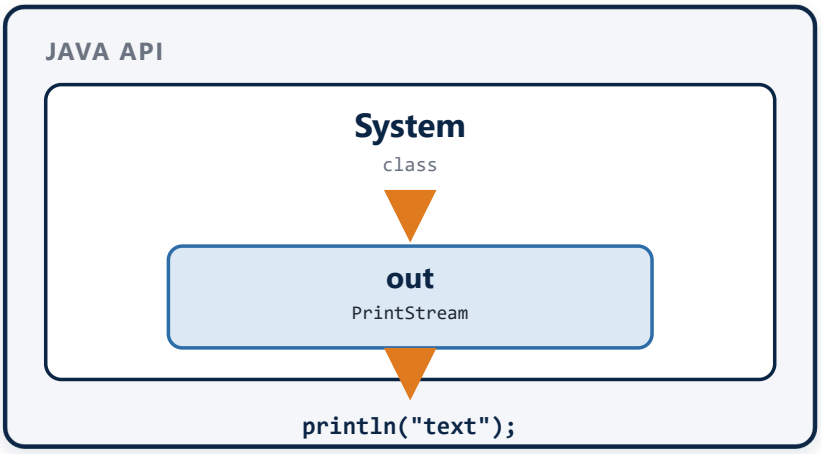
- **Compile:** check the Java source and create bytecode.
- **Run:** execute the compiled bytecode on the Java Virtual Machine.

In IntelliJ IDEA, the Run command performs these steps through the project interface and displays the result in the Run window.

Reading System.out.println

The dots show a path through Java's API:

- `System` identifies a class in the Java API;
- `out` identifies the standard output `PrintStream`; and
- `println` identifies the method being called.



Reading System.out.println from the System class to the println method

print and println

Both methods display output, but they leave the cursor in different positions.

Method	What happens afterwards
`System.out.print(...)`	The cursor remains at the end of the current line.
`System.out.println(...)`	The cursor moves to the beginning of the next line.

Using only `print` joins the messages on one line:

```
System.out.print("I love the following:");
System.out.print("Sweets");
System.out.print("Chips");
System.out.print("Coffee");
```

CONSOLE OUTPUT

```
I love the following:SweetsChipsCoffee
```

Using `println` inserts line breaks. The final `print` leaves the cursor after `Coffee`.

```
System.out.println("I love the following:");
System.out.println("Sweets");
System.out.println("Chips");
System.out.print("Coffee");
```

CONSOLE OUTPUT

```
I love the following:
Sweets
Chips
Coffee|
```

The vertical bar represents the cursor; it is not part of the program's output.

The newline escape sequence

An **escape sequence** begins with a backslash followed by a control character. The sequence `\n` represents a new-line character inside a string.

```
System.out.print("I love the following:\n");
System.out.print("Sweets\n");
System.out.print("Chips\n");
System.out.print("Coffee\n");
```

Although every call uses `print`, each string contains its own line break:

CONSOLE OUTPUT

```
I love the following:
Sweets
Chips
Coffee
|
```

Variables, literals and assignment

A **variable** gives a program a named place to store a value. A **literal** is a value written directly in the source code.

```java
public class Variables {
    public static void main(String[] args) {
        int value;                          // declaration
        value = 10;                         // assignment
        System.out.print("The value is "); // string literal
        System.out.println(value);          // stored value
    }
}
```

CONSOLE OUTPUT

```
The value is 10
```

The declaration `int value;` tells the compiler the variable's name and the type of data it can hold. The assignment `value = 10;` means "store 10 in `value`". A variable must be declared before it is used.

When a variable is sent to `print` or `println`, Java displays its current value. A variable name is written without quotation marks; text written inside quotation marks is a **string literal**.

Joining text and values

When + has a string operand, it performs **string concatenation**. To concatenate means to append one item to another.

```java
System.out.println("Today is " + "a good day!");

int number = 857623;
System.out.println("Today's lotto number is: " + number);
```

CONSOLE OUTPUT

```
Today is a good day!
Today's lotto number is: 857623
```

Java converts the numeric value to text while it builds the output.

Identifier rules and conventions

An **identifier** is a name for a program element, such as a variable or class.

- The first character must be a letter, underscore (_) or dollar sign ($).
- Later characters may also include digits.
- Spaces are not permitted.
- Uppercase and lowercase are different: `numberOfCows` and `numberofcows` are different identifiers.
- Choose names that communicate purpose rather than vague names such as `y`.

Java naming convention uses **camelCase** for variables and **PascalCase** for classes:

```
int numberOfCows;

class FarmCows {
}
```

What a type controls

A variable's data type determines what it can store, how Java represents and processes that value, and how much memory is used.

- Whole numbers use integer types such as `int` and `long`.
- Decimal values use floating-point types such as `float` and `double`.
- A `char` stores one character.
- A `boolean` stores either `true` or `false`.

The type must be compatible with the value assigned to the variable.

Data type	Size	Values
`byte`	1 byte	Integers from −128 to +127
`short`	2 bytes	Integers from −32,768 to +32,767
`int`	4 bytes	Integers from −2,147,483,648 to +2,147,483,647
`long`	8 bytes	Very large whole numbers
`float`	4 bytes	Floating-point values, about 7 digits of accuracy
`double`	8 bytes	Floating-point values, about 15 digits of accuracy

Integer types

The integer types are `byte`, `short`, `int` and `long`. Java treats an integer literal such as `14` as an `int` by default; add `L` when a literal must be treated as a `long`.

```
long numberOfCows = 14;
long anotherNumber = 14L;
```

Example: declaring integer values

This complete example declares one variable of each integer type and inserts their values into output strings.

```
public class IntegerVariables {
    public static void main(String[] args) {
        byte miles = 120;
        short minutes = 100;
        int temperatureLondon = -15;
        long days = 182500;

        System.out.println(
            "We have made a journey of "
                + miles + " miles in only "
                + minutes + " minutes.");
        System.out.println(
            "The temperature in London is "
                + temperatureLondon + " degrees.");
        System.out.println(
            "There are " + days + " days in 500 years.");
    }
}
```

CONSOLE OUTPUT

```
We have made a journey of 120 miles in only 100 minutes.
The temperature in London is -15 degrees.
There are 182500 days in 500 years.
```

The variable's type controls its storage; concatenation controls how its value is displayed. Java reads a concatenation from left to right, looks up each variable's current value, converts it to text and appends it to the growing string.

Floating-point types

The floating-point types are `float` and `double`. Java treats a decimal literal as a `double` by default, so the suffix `F` is required when the literal must be a `float`.

```
float pay = 1800.99;    // error: the literal is a double
float salary = 1800.99F;
```

Boolean values

A `boolean` stores one of two logical states:

Value	Meaning
`true`	A condition is satisfied.
`false`	A condition is not satisfied.

```
boolean value = true;
System.out.println(value);

value = false;
System.out.println(value);
```

CONSOLE OUTPUT

```
true
false
```

A boolean is not a number. It is useful when a program must evaluate a condition and choose between alternatives.

Characters and Unicode

A `char` stores one character. Character literals use **single quotation marks**, while a `String` stores a sequence of characters in double quotation marks.

```
char letter = 'A';
System.out.println(letter);

letter = 'B';
System.out.println(letter);
```

CONSOLE OUTPUT

```
A
B
```

Characters are internally represented by numeric codes. Java uses **Unicode**, which provides codes for characters from many writing systems. A Java `char` occupies two bytes and represents a UTF-16 code unit.

Operators and operands

An **operator** performs an operation; its **operands** are the values on which it acts. Unary operators use one operand, binary operators use two, and ternary operators use three. The introductory arithmetic operators are binary.

Operator	Meaning	Example
`+`	Addition	`5 + 4` produces `9`
`-`	Subtraction	`5 - 4` produces `1`
`*`	Multiplication	`5 * 4` produces `20`
`/`	Division	`20 / 4` produces `5`
`%`	Modulus: remainder after division	`22 % 4` produces `2`

```
answer = 5 + 4;
pay = salary - tax;
pay = hours * rate;
average = marks / number;
leftOver = 22 % 4;
```

Use *, not the multiplication symbol ×, in Java source. The order of operands matters for subtraction and division. Modulus is especially useful for checking whether a number divides evenly.

Integer and floating-point division

When both operands are integers, Java performs **integer division** and discards any fractional part. When at least one operand is floating point, the result can contain a fraction.

```
double parts;
parts = 22 / 4;      // 5.0: integer division happens first
parts = 22.0 / 4;    // 5.5
```

Operator precedence

When an expression contains several operators, Java evaluates them in this order:

1. parentheses;
2. multiplication, division and modulus; then
3. addition and subtraction.

```
int result;
result = 2 + 3 * 4;       // 14
result = (2 + 3) * 4;     // 20
```

Use parentheses whenever they make the intended calculation clearer, even if Java's precedence rules would already produce the correct result.

CHAPTER 03

Week 2 · Output and calculations

Writing Java programs and working with values

From source code to output

A Java program begins as source code in a `.java` file. The compiler checks the source and translates it into portable **bytecode** in a `.class` file. The Java Virtual Machine then runs that bytecode.

Stage	What happens
Write	Create Java source code in IntelliJ IDEA.
Compile	Check the source and translate it into bytecode.
Run	Ask the Java Virtual Machine to execute the bytecode.
Output	View the program's result in the console.

Displaying output

Lab 1 concentrates on the structure of a small Java program and precise control of its console output.

A small Java program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

The program contains several important parts:

- the **class** named `Hello` contains the program;
- the **main method** is where execution begins;
- braces `{ }` mark the class and method bodies;
- `System.out.println("Hello");` is a **statement**; and
- the semicolon marks the end of that statement.

The public class name and filename must match. The class `Hello` must therefore be saved as `Hello.java`.

Learning from compiler errors

Lab 1 deliberately asks you to break working code and repair it. This is useful because compiler messages become easier to understand with practice.

Use a controlled process:

1. Begin with a program that compiles.
2. Make one change at a time.
3. Compile again and read the first error message.
4. Inspect the named line and the line immediately before it.
5. Repair the program before trying another change.

Common causes this week include a missing brace or semicolon, incorrect capitalisation such as `Println`, a misspelled `main`, and a class name that does not match the filename.

Java is case-sensitive

`println`, `Println` and `PRINTLN` are three different names. Only the correctly capitalised form is recognised.

print and println

Both methods display information, but they leave the console cursor in different places.

```
System.out.println("First");
System.out.println("Second");
```

CONSOLE OUTPUT

```
First
Second
```

`println` starts a new line after displaying its value.

```
System.out.print("First ");
System.out.print("Second");
```

CONSOLE OUTPUT

```
First Second
```

`print` leaves the cursor on the same line. The space after `First` is part of the String literal.

Escape sequences

An **escape sequence** begins with a backslash and represents a character that is awkward to type directly inside a String.

Sequence	Meaning
`\n`	Start a new line.
`\t`	Insert a horizontal tab.
`\"`	Print a double quotation mark.
`\\`	Print one backslash.

```
System.out.print("Name\tMark\nAna\t72\n");
System.out.println("She said \"Hello\".");
System.out.println("C:\\Java\\Week2");
```

CONSOLE OUTPUT

```
Name    Mark
Ana     72
She said "Hello".
C:\Java\Week2
```

The exact width of a tab can vary. Check the console rather than counting spaces in the source code.

Variables and types

Lab 2 introduces named values and the types Java uses to represent different kinds of data.

Variables and literals

A **variable** is a named place to store a value. A **literal** is a value written directly in source code.

```
int score;
score = 72;
System.out.println(score);
```

The first statement declares an integer variable named `score`. The second assigns the integer literal `72`. The third reads and prints the stored value.

Declaration and initial assignment can also be written together:

```
int score = 72;
```

The variable can receive a different value later because it is not a constant.

Identifiers and naming

An **identifier** is a programmer-chosen name for a class, method, variable or other program element.

An identifier:

- may contain letters, digits, _ and $;
- cannot begin with a digit;
- cannot contain spaces;
- cannot be a Java keyword such as `class` or `int`; and
- is case-sensitive, so `score` and `Score` are different names.

Use names that explain what the value represents.

What is named?	Convention	Example
Variable	**camelCase**	`hoursWorked`
Class	**PascalCase**	`WeeklyPay`
Constant	Uppercase with underscores	`TAX_RATE`

Primitive data types

A **primitive type** represents one simple value directly.

Kind of value	Java types
Whole numbers	`byte`, `short`, `int`, `long`
Decimal numbers	`float`, `double`
One character	`char`
True or false	`boolean`

Whole-number types

Type	Size	Approximate range
`byte`	8 bits	−128 to 127
`short`	16 bits	−32 thousand to 32 thousand
`int`	32 bits	−2.1 billion to 2.1 billion
`long`	64 bits	much larger whole numbers

Use `int` for ordinary whole-number calculations unless there is a clear reason to choose another type.

Decimal, character and true-or-false values

```
float temperature = 18.5F;
double price = 24.99;
char grade = 'A';
boolean complete = false;
```

- A `float` literal needs an `F` suffix.
- Java treats a decimal literal as `double` by default.
- A `char` stores one character inside single quotation marks.
- A `boolean` stores only `true` or `false`.

String variables

A **String** stores text made from zero or more characters. A **String literal** is enclosed in double quotation marks.

```
String course = "Software Design";
System.out.println(course);
```

A String is not one of Java's eight primitive types, but it is used frequently to store and display text.

String concatenation

When one operand is a String, + performs **string concatenation**: it joins the values as text.

```
String name = "Ana";
int mark = 72;

System.out.println(name + " scored " + mark);
```

CONSOLE OUTPUT

```
Ana scored 72
```

Calculations

Lab 3 uses stored values to calculate results and display them with meaningful labels.

Arithmetic operators

An **operator** acts on values called **operands**.

Operator	Operation	Example
+	addition	`total = first + second;`
-	subtraction	`change = paid - cost;`
*	multiplication	`area = width * height;`
/	division	`average = total / count;`
%	remainder	`leftOver = total % groupSize;`

Order of operations

Java evaluates arithmetic expressions in this order:

1. parentheses;
2. multiplication, division and modulus, from left to right;
3. addition and subtraction, from left to right.

```
int first = 10 + 2 * 3;     // 16
int second = (10 + 2) * 3;  // 36
```

Parentheses change the first operation and make the formula's intention clearer.

Integer division and modulus

When both operands are integers, Java performs **integer division** and discards the fractional remainder.

```
int whole = 22 / 4;       // 5
double exact = 22.0 / 4;  // 5.5
```

The modulus operator `%` produces the remainder:

```
int groups = 22 / 4;     // 5 complete groups
int leftOver = 22 % 4;   // 2 left over
```

Percentages

Write a percentage as a decimal before using it in a calculation.

Percentage	Decimal
50%	`0.50`
21%	`0.21`
14%	`0.14`
3%	`0.03`

```
double surveySize = 12467;
double buyers = surveySize * 0.14;
```

Constants with final

A **constant** is a named value that must not change while the program runs. Declare one with `final` and conventionally write its name in uppercase with underscores.

```
final double TAX_RATE = 0.21;
double wage = 450.0;
double tax = wage * TAX_RATE;
```

The name `TAX_RATE` explains the formula more clearly than an unexplained literal `0.21`.

Arithmetic with byte and short

Java performs arithmetic on `byte` and `short` values as `int`. Store the result in an `int` variable.

```
byte first = 10;
byte second = 20;
int total = first + second;
```

No manual conversion syntax is required for this week's labs.

Calculate before concatenating

Parentheses are important when a calculation must happen before its result is joined to a message.

```
int first = 10;
int second = 20;

System.out.println("Total: " + (first + second));
System.out.println("Values: " + first + second);
```

CONSOLE OUTPUT

```
Total: 30
Values: 1020
```

In the first statement, parentheses force the addition to happen first. In the second, Java begins concatenating from left to right and joins `10` followed by `20`.

Single-line comments

A comment beginning with `//` continues to the end of its line. Java ignores it when compiling and running the program.

```
// Calculate the tax, not the final wage
double tax = wage * TAX_RATE;

double area = length * width; // square metres
```

Use comments to explain intent or a decision that is not already obvious from a good variable name.

A complete calculation

```
public class WeeklyPay {
    public static void main(String[] args) {
        final double TAX_RATE = 0.21;
        double grossPay = 450.0;
        double tax = grossPay * TAX_RATE;
        double netPay = grossPay - tax;

        System.out.println("Gross pay: " + grossPay);
        System.out.println("Tax: " + tax);
        System.out.println("Net pay: " + netPay);
    }
}
```

This program follows a useful pattern: declare the values, perform the calculations, and then display clearly labelled results.

Unit conversions with a named factor

A conversion is a multiplication or division by a factor with a clear meaning. Write the formula before choosing the Java expression, and label the units in the output.

```
final double CENTIMETRES_PER_METRE = 100.0;
double metres = 0.85;
double centimetres = metres * CENTIMETRES_PER_METRE;
System.out.println("Centimetres: " + centimetres);
```

The result is 85.0 centimetres. Multiplication converts metres to centimetres here; dividing by the same factor reverses that conversion. Keep the input and output units distinct rather than naming every variable `number`.

Whole units and remainders

Integer division finds the number of complete groups. Modulus finds what remains. This example packs objects into boxes of equal size.

```
int objects = 17;
int fullBoxes = objects / 6;
int remaining = objects % 6;
System.out.println(fullBoxes + " full boxes; " + remaining + " left over");
```

CONSOLE OUTPUT

```
2 full boxes; 5 left over
```

Check 0, 5, 6 and 7 as well as 17. Testing the boundary at one full box helps expose the difference between a complete group and a remainder. Apply that distinction when a lab asks you to split a quantity into larger units and leftover smaller units.

Comments and a calculation plan

A `//` comment ends at the end of its line. A block comment begins with `/*` and ends with `*/`. Block comments cannot be nested.

```
/* Convert the starting distance before displaying it.
   Keep the conversion factor in one named constant. */
final double METRES_PER_KM = 1000.0;
double kilometres = 2.5;
double metres = kilometres * METRES_PER_KM;
System.out.println(metres);
```

For each small calculation, identify the given values, the required result, the formula and the output labels. Predict one answer before running the program. Change one value and calculate the new expected result; storing a pre-calculated answer would not work for the second input.

CHAPTER 04

Week 3 · Input and decisions

Scanner input, comparisons and if statements

Learning outcomes

After studying this chapter, you should be able to:

- create a `Scanner` connected to the keyboard;
- read text and numeric values with the appropriate Scanner method;
- explain why `nextLine()` can appear to be skipped after a numeric read;
- form boolean expressions with relational operators; and
- use `if`, `if-else` and nested decisions to control a program.

Importing and creating a Scanner

`System.in` represents the standard input device, which is normally the keyboard. A `Scanner` object makes keyboard input easier to read as text and Java values.

Add this import before the class declaration:

```
import java.util.Scanner;
```

Then create a Scanner in `main`:

```
Scanner keyboard = new Scanner(System.in);
```

The declaration gives the Scanner the name `keyboard`. The expression `new Scanner(System.in)` creates an object connected to keyboard input.

Choosing a Scanner method

Use a method that matches the kind of value the program needs.

Method	Value returned	Example
`nextByte()`	`byte`	`byte count = keyboard.nextByte();`
`nextShort()`	`short`	`short year = keyboard.nextShort();`
`nextInt()`	`int`	`int age = keyboard.nextInt();`
`nextLong()`	`long`	`long distance = keyboard.nextLong();`
`nextFloat()`	`float`	`float rate = keyboard.nextFloat();`
`nextDouble()`	`double`	`double price = keyboard.nextDouble();`
`next()`	one token as a `String`	`String word = keyboard.next();`
`nextLine()`	the rest of the line as a `String`	`String name = keyboard.nextLine();`

For example, a prompt followed by `nextInt()` reads an integer and stores it in a variable:

```
int age;
Scanner keyboard = new Scanner(System.in);

System.out.print("Enter your age: ");
age = keyboard.nextInt();
```

`next()` stops at whitespace, so it reads one word. `nextLine()` reads the whole line, including spaces, up to the Enter key.

Reading a line after a number

Numeric Scanner methods such as `nextInt()` and `nextDouble()` read the number but leave the Enter key's newline in the input. If `nextLine()` is called immediately afterwards, it consumes that leftover newline and returns an empty string.

Consume the leftover newline before asking for a line of text:

```
int age;
String name;
Scanner keyboard = new Scanner(System.in);

System.out.print("What is your age? ");
age = keyboard.nextInt();
keyboard.nextLine(); // consume the newline after the number

System.out.print("What is your name? ");
name = keyboard.nextLine();

System.out.println("Hello " + name + ". You are " + age + ".");
```

The first `nextLine()` after the number clears the remainder of that input line. The second one reads the name.

Reading one character

Scanner has no `nextChar()` method. Read a token as a `String` and take its first character with `charAt(0)`:

```
Scanner keyboard = new Scanner(System.in);
System.out.print("Enter a character: ");
char letter = keyboard.next().charAt(0);
```

A complete input-and-calculation example

Use the same source-file structure as your earlier programs, with the `Scanner` import added above the class. This small illustration converts a measurement in metres to centimetres. Save it as `ReadLength.java`.

```
import java.util.Scanner;

public class ReadLength {
    public static void main(String[] args) {
        final double CENTIMETRES_PER_METRE = 100.0;
        Scanner keyboard = new Scanner(System.in);
        System.out.print("Length in metres: ");
        double metres = keyboard.nextDouble();
        double centimetres = metres * CENTIMETRES_PER_METRE;
        System.out.println("Centimetres: " + centimetres);
    }
}
```

Typing 0.85 gives 85.0 centimetres. The prompt is displayed before the read; `nextDouble()` waits for input, and the following statements use the value entered. Zero produces zero. A word cannot be read by `nextDouble()` as a number; this example assumes numeric input.

Boolean expressions and relational operators

A boolean expression evaluates to either `true` or `false`. Relational operators compare two values and produce a boolean result.

Operator	Meaning	Example question
`>`	greater than	Is `mark > 50`?
`<`	less than	Is `temperature < 0`?
`>=`	greater than or equal to	Is `age >= 18`?
`<=`	less than or equal to	Is `price <= budget`?
`==`	equal to	Is `number == 10`?
`!=`	not equal to	Is `number != 0`?

In Java, `=` assigns a value. `==` compares two values. Keep those operators distinct.

The if statement

An `if` statement runs its controlled statement only when its boolean expression is true.

```
double temperature = -2.0;
if (temperature < 0) {
    System.out.println("Water may freeze.");
}
```

When the condition is false, Java skips the block and continues with the statement after it. Curly braces make the block clear and allow several statements to depend on the same condition.

```
double average = 97.0;
boolean highScore = false;
if (average > 95) {
    highScore = true;
    System.out.println("That's a high score!");
}
```

Without braces, an `if` controls only the next statement. Indent the controlled statement, and use braces for blocks so the program's structure is easy to read.

Using input in a decision

This complete program reads an integer and compares it with 10:

```
import java.util.Scanner;

public class IfNumbers {
    public static void main(String[] args) {
        Scanner keyboard = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = keyboard.nextInt();

        if (number > 10) {
            System.out.println("The number is greater than 10.");
        }
        if (number < 10) {
            System.out.println("The number is less than 10.");
        }
        if (number == 10) {
            System.out.println("The number is 10.");
        }
    }
}
```

Each `if` is tested independently. For an input of `10`, only the final condition is true, so the output is:

CONSOLE OUTPUT

```
Enter a number: 10
The number is 10.
```

Characters can also be compared. Java stores each `char` as a Unicode value, so relational operators can test their order:

```java
char letter = 'A';
if (letter < 'Z') {
    System.out.println("A comes before Z.");
}
```

Choosing between two actions with if-else

Use `if-else` when exactly one of two alternatives should run. The `else` branch runs when the condition is false.

```java
double number1 = 12.0;
double number2 = 5.0;
if (number2 == 0) {
    System.out.println("Division by zero is not possible.");
} else {
    double quotient = number1 / number2;
    System.out.println("The quotient is " + quotient);
}
```

Here, the division happens only in the `else` block, after the program has established that the divisor is not zero.

With these values, the quotient is 2.4. Change the divisor to 0.0 and confirm that the message appears instead. If both operands had been `int`, their division would have used integer division even if the result were stored in a `double`.

Testing the edges of a decision

For a threshold of 10, use values 9, 10 and 11. They exercise the below, equal and above cases. For a valid range of 0 to 100, also test -1 and 101 so you can check the rejected cases.

Mistake	What to check
A matching value takes the wrong branch	Did you use > when the boundary needs >=?
A statement always runs	Is there a stray semicolon after the `if` condition?
Only the first statement depends on the condition	Put all controlled statements inside braces.
A calculation uses the wrong type	Check the types of the operands, not only the result variable.
A typed name loses its second word	Use `nextLine()` for a whole line rather than `next()`.

Nested decisions

An `if` inside another `if` is a nested decision. Java reaches the inner decision only when the outer condition allows it to continue.

```
boolean coldOutside = true;
boolean snowing = false;
if (coldOutside) {
    if (snowing) {
        System.out.println("Wear a parka.");
    } else {
        System.out.println("Wear a jacket.");
    }
} else {
    System.out.println("A light outfit should be fine.");
}
```

If it is not cold, the outer `else` runs. If it is cold, Java checks whether it is snowing and chooses between the inner branches.

An `else` belongs to the nearest preceding `if` that does not already have an `else`. Braces and consistent indentation make that relationship visible.

Classifying a mark

This nested decision distinguishes a fail, a pass and a distinction:

```
int mark = 72;
if (mark < 50) {
    System.out.println("You failed.");
} else {
    if (mark >= 80) {
        System.out.println("You passed with distinction.");
    } else {
        System.out.println("You passed.");
    }
}
```

When `mark` is below 50, the first branch runs. Otherwise, the inner decision separates marks of 80 or more from the remaining passing marks.

CHAPTER 05

Week 4 · More decisions

Logical operators, strings and switch

Learning outcomes

After studying this chapter, you should be able to:

- choose one branch from several with `else if`;
- combine and reverse conditions with `&&`, `||` and `!`;
- compare `String` values by content rather than by reference; and
- select a menu action with a `switch` statement.

Choosing among several alternatives

An `if-else-if` chain tests conditions from top to bottom. Java runs the first branch whose condition is true, then skips the rest. The final `else` handles every case that did not match.

```
int mark = 72;

if (mark >= 80) {
    System.out.println("Distinction");
} else if (mark >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

The output is `Pass`. By the time Java reaches `mark >= 50`, it already knows the mark is below 80. Put the highest threshold first; reversing these two tests would classify 80 or above as a pass.

Use separate `if` statements when several actions may all be needed. Use an `if-else-if` chain when exactly one branch should run.

Combining conditions

Operator	Meaning	Example that is true
`&&`	Both conditions must be true	`age >= 18 && age <= 65` when age is 30
`||`	At least one condition must be true	`day == 6 || day == 7` when day is 6
`!`	Reverse a boolean value	`!finished` when `finished` is `false`

To check whether a mark is between 0 and 100 inclusive, write both comparisons:

```
int mark = 72;
if (mark >= 0 && mark <= 100) {
    System.out.println("Valid mark");
} else {
    System.out.println("Mark must be between 0 and 100");
}
```

Java does not interpret `0 <= mark <= 100` as a mathematical range. The first comparison produces a boolean, which cannot then be compared with `100`.

Java evaluates `&&` and `||` from left to right and stops once the result is known. In `divisor != 0 && total / divisor > 10`, the division is skipped when `divisor` is zero. Parentheses make longer conditions easier to check, even where precedence would give the same result.

Comparing strings

A `String` is an object. `==` asks whether two references identify the same object; it is unsuitable for checking whether two separately entered pieces of text contain the same characters. Use `.equals()` for an exact comparison or `.equalsIgnoreCase()` when capitalisation should not matter.

```
Scanner keyboard = new Scanner(System.in);
String answer = keyboard.nextLine();

if (answer.equalsIgnoreCase("yes")) {
    System.out.println("Continuing");
} else {
    System.out.println("Stopping");
}
```

`"yes".equalsIgnoreCase(answer)` is also valid. If `answer` might be `null`, putting the known literal first avoids a `NullPointerException`.

The comparison methods return a `boolean`, so they can be used directly as `if` conditions. For alphabetical ordering, `compareTo()` returns a negative value, zero or a positive value. It does not return a boolean:

```
String firstName = "Anna";
String secondName = "Ben";
if (firstName.compareTo(secondName) < 0) {
    System.out.println(firstName + " comes first");
}
```

Choosing from a menu with switch

A traditional `switch` is useful when one value is compared with several exact alternatives. Each `case` has a label. `break` leaves the switch after the chosen action, and `default` handles an unrecognised value.

```
import java.util.Scanner;

public class MenuChoice {
    public static void main(String[] args) {
        Scanner keyboard = new Scanner(System.in);
        System.out.print("Choose 1 for tea or 2 for coffee: ");
        int choice = keyboard.nextInt();

        switch (choice) {
            case 1:
                System.out.println("Tea selected");
                break;
            case 2:
                System.out.println("Coffee selected");
                break;
            default:
                System.out.println("Unknown choice");
        }
    }
}
```

If `break` is omitted, execution continues into the next case. That is called *fall-through* and can be intentional, but it is usually a mistake in a simple menu. A `switch` checks exact values, not ranges such as `mark >= 50`; use `if-else-if` for ranges.

A truth table for logical operators

&& requires both conditions. || requires at least one. ! reverses a boolean. A truth table lets you check every possible pair without guessing from one input.

A	B	A && B	A || B	!A
false	false	false	false	true
false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

`age >= 18 && hasTicket` asks whether both facts are true. `choice == 'Y' || choice == 'y'` accepts either spelling of one response. Write a complete comparison on each side: `choice == 'Y' || 'y'` is not a valid boolean expression.

Inside a range, outside a range

An inclusive range needs an AND. Being outside it needs an OR.

```
int mark = 72;
boolean inside = mark >= 0 && mark <= 100;
boolean outside = mark < 0 || mark > 100;
System.out.println(inside);
System.out.println(outside);
```

CONSOLE OUTPUT

```
true
false
```

Try -1, 0, 100 and 101. The lower and upper accepted endpoints belong to the inside case. `mark < 0 && mark > 100` can never be true: one number cannot be below zero and above 100 at the same time.

Grouping and short-circuit evaluation

Use parentheses to make the intended groups clear. Java applies ! before &&, and && before ||.

```
int age = 20;
boolean student = false;
boolean member = true;
boolean discount = age < 18 || (student && member);
System.out.println(discount);
```

This prints false. Membership alone is not enough: the right-hand group requires both student and member. With `(age < 18 || student) && member`, membership would be required for everyone.

Short-circuiting also protects an operation: `text.length() > 0 && text.charAt(0) == 'A'` checks length first, so it avoids an invalid character access for empty text.

Exact text, ignored case and ordering

Choose the comparison method that answers your question. String comparison methods examine text content; == compares references.

Method	Result	Use
`first.equals(second)`	`boolean`	Same characters and same case.
`first.equalsIgnoreCase(second)`	`boolean`	Same text when case should not matter.
`first.compareTo(second)`	Negative, zero or positive `int`	Case-sensitive ordering.
`first.compareToIgnoreCase(second)`	Negative, zero or positive `int`	Ordering with case ignored.

`compareTo` does not promise exactly -1 or 1. Test `< 0`, `== 0` or `> 0`. Its ordering is based on character values, so do not assume it gives a language-specific dictionary order for every name or symbol.

Reading a String comparison result

```
String first = "apple";
String second = "pear";
int order = first.compareTo(second);
if (order < 0) {
    System.out.println(first + " comes before " + second);
} else if (order == 0) {
    System.out.println("The words match.");
} else {
    System.out.println(second + " comes before " + first);
}
```

The output is `apple comes before pear`. Test matching words and words whose first letters differ only in case. Use `compareToIgnoreCase` when the question specifies that case should be ignored.

Switching on a character or String

A traditional `switch` can use an integer, a `char`, or a `String` in these labs. A `char` case uses single quotes; a String case uses double quotes. String cases match text exactly, including case.

```
char answer = 'y';
switch (answer) {
    case 'Y':
    case 'y':
        System.out.println("Continue");
        break;
    case 'N':
    case 'n':
        System.out.println("Stop");
        break;
    default:
        System.out.println("Please enter Y or N.");
}
```

Adjacent case labels can deliberately share one action. Here Y and y both reach `Continue`. `break` exits the switch, while `default` gives a useful response to an unsupported choice.

Selecting a rate, then calculating

A choice can select a value used in a later calculation. Keep selection and calculation separate so each is easy to check. This small example illustrates using an already-selected rate.

```
double litresPerMinute = 2.0;
double minutes = 3.5;
System.out.println("Litres: " + litresPerMinute * minutes);
```

This displays `Litres: 7.0`. If a menu selected a flow setting, the switch could choose the rate and the later expression could use it. Make sure every permitted branch supplies a value before it is used. Test the default case too.

Choosing the right decision structure

Requirement	Useful structure
An action may or may not happen	`if`.
Exactly one of two alternatives	`if-else`.
One of several ranges	`if-else-if`, with thresholds in the right order.
Several independent actions may apply	Separate `if` statements.
One value selects an exact menu option	`switch`.

For a calculator, check a zero divisor in the division branch before calculating. For marks, check the valid input range before assigning a grade. For a converter, label the original and converted units. These checks belong to the chosen operation, rather than an unrelated branch.

Check your understanding

1. In the mark example, what happens if `mark` is exactly 80? Why?
2. Write a condition for an age outside the range 18 to 65 inclusive.
3. Why should a program use `answer.equals("yes")` rather than `answer == "yes"` for keyboard input?
4. What is printed if the menu user enters `9`?

End of Week 4

CHAPTER 06

Week 5 · Loops and formatting

Repetition, validation and formatted output

Learning outcomes

After studying this chapter, you should be able to:

- update a counter with increment, decrement and combined assignment;
- trace and write a `while` loop that terminates;
- use `do-while` when the work must happen at least once;
- use a sentinel to stop an input loop and a condition to validate input; and
- format numbers and text with `System.out.printf()`.

Counters and updates

`count++` adds one to `count`; `count--` subtracts one. `count += 3` adds three. These change the stored value, unlike `count + 1`, which merely calculates a new value unless it is assigned.

```
int count = 0;
count++;      // count is now 1
count += 3;   // count is now 4
count--;      // count is now 3
```

In a larger expression, `count++` returns the old value and `++count` returns the new value. For now, put an update on its own line so its effect is clear.

The while loop

A `while` loop checks its condition *before* each iteration. If the condition is false at the first check, the body runs zero times.

```
int number = 1;

while (number <= 5) {
    System.out.println(number);
    number++;
}
```

This prints 1 through 5. After printing 5, `number` becomes 6; the next condition is false. To trace a loop, record the value at the condition, the body action and the update:

Check	`number <= 5`	Printed	Value after update
1	true, at 1	1	2
2	true, at 2	2	3
3–5	true	3, 4, 5	6
6	false, at 6	nothing	6

Every loop needs a route to a false condition. Omitting `number++` here would print `1` indefinitely. An off-by-one error comes from using the wrong boundary: `number < 5` prints only 1 through 4.

Running totals and averages

A *running total* starts at zero and grows as values are processed. A counter records how many values were processed.

```
int total = 0;
int count = 1;

while (count <= 4) {
    total += count;
    count++;
}

System.out.println("Total: " + total); // 10
```

For an average, divide the total by the number of values. Cast one operand to `double` if a fractional result is needed: `double average = (double) total / numberOfValues;`. Do not divide if the number of values is zero.

Sentinel-controlled input

A *sentinel* is a special input value that means “stop”; it must not be included in the calculation. This program adds entered integers until the user types `-1`:

```java
import java.util.Scanner;

public class SentinelTotal {
    public static void main(String[] args) {
        Scanner keyboard = new Scanner(System.in);
        int total = 0;

        System.out.print("Enter a number (-1 to stop): ");
        int number = keyboard.nextInt();

        while (number != -1) {
            total += number;
            System.out.print("Enter a number (-1 to stop): ");
            number = keyboard.nextInt();
        }

        System.out.println("Total: " + total);
    }
}
```

The first read occurs before the condition so the loop has a value to test. The next read occurs inside the loop so a new value can eventually end it.

Input validation

Repeat a prompt while the value is outside the accepted range. An invalid number is below 1 **or** above 100:

```java
Scanner keyboard = new Scanner(System.in);
System.out.print("Enter a number from 1 to 100: ");
int number = keyboard.nextInt();

while (number < 1 || number > 100) {
    System.out.print("Out of range. Try again: ");
    number = keyboard.nextInt();
}

System.out.println("Accepted: " + number);
```

This validates the *range* of an integer. Handling text that cannot be read by `nextInt()` needs a separate input strategy.

Formatting output with printf

`printf` prints a format string with placeholders for values. `%d` is an integer, `%s` is text, `%f` is a decimal number and `%.2f` displays two digits after the decimal point. Unlike `println`, `printf` does not automatically add a newline; use `%n`.

```
String item = "Notebook";
int quantity = 3;
double price = 2.5;

System.out.printf("%-12s %3d  €%6.2f%n", item, quantity, price);
```

The item is left aligned in 12 characters, the quantity is right aligned in 3, and the price is shown with two decimal places in a width of 6. The `€` is ordinary text. Match each placeholder to a value of the appropriate type. Formatting rounds the *displayed* value; it does not change the value stored in `price`.

Prefix and postfix updates

On their own lines, `number++` and `++number` both increase the variable by one. Inside a larger expression, they supply different values to that expression.

```
int number = 10;
System.out.println(number++);
System.out.println(number);
System.out.println(++number);
```

CONSOLE OUTPUT

```
10
11
12
```

Postfix supplies the old value and then updates the variable. Prefix updates first and supplies the new value. Decrement has the same distinction with `--`. Keep updates separate while learning a new loop; combine them with output only when you can predict the result.

Counting forwards, backwards and in steps

A counted `while` loop needs a starting value, a boundary test and an update that eventually reaches that boundary.

Task	Start	Condition	Update
1 through 5	`number = 1`	`number <= 5`	`number++`
5 down to 1	`number = 5`	`number >= 1`	`number--`
Odd numbers 1 through 9	`number = 1`	`number <= 9`	`number += 2`

For a user-supplied range, work through the first and last permitted values. When the question asks for ascending output, establish which entered value is the lower bound before counting.

Squares, cubes and products

Use the loop variable in each calculation, so the output changes on every pass. This fragment prints the squares of 1 through 3:

```
int number = 1;
while (number <= 3) {
    System.out.println(number * number);
    number++;
}
```

The output is 1, 4 and 9, each on a separate line. A cube uses `number * number * number`.

A sum starts at **zero**. A product starts at **one**, because multiplying a zero total would leave it at zero forever. Larger products can exceed an integer type's range; choose a short trace to check your starting value and updates.

Conditions inside a loop

Use a loop for repetition and an `if` for a choice on each pass. The update must still happen whether or not the condition matches.

```
int number = 1;
while (number <= 7) {
    if (number % 2 != 0) {
        System.out.println(number);
    }
    number++;
}
```

This prints 1, 3, 5 and 7. `number % 5 == 0` tests divisibility by 5. Test the written condition rather than guessing from the name of an exercise.

In the Week 5 **FizzBuzzBasic** exercise, only the exact values 3, 5 and 15 are replaced with words; the other values print normally. That question uses equality checks, rather than replacing every multiple of 3 or 5.

Letting the user choose a repetition count

The requested number of repetitions and the number already performed are different quantities. Read the requested limit before beginning a counted loop, then use a separate counter to track progress.

Decide whether your counter starts at zero or one. Work through a request for zero repetitions and a request for one repetition before choosing the comparison operator. Check that the update moves towards the stopping point.

When the repeated output includes text entered by the user, choose a Scanner method that preserves the text you need. Revisit the Week 3 input notes if the text can contain spaces. Write your own loop by adapting the counter pattern, then trace it with a small limit.

The do-while loop

A `do-while` loop tests its condition **after** running the body. It is useful when a question must be asked at least once.

```
int number = 3;
do {
    System.out.println(number);
    number--;
} while (number > 0);
```

CONSOLE OUTPUT

```
3
2
1
```

The final `while` needs a semicolon. An ordinary `while` does not have a semicolon immediately after its condition. With a starting value of zero, this `do-while` still prints once; the equivalent `while (number > 0)` would not print at all.

Repeating until the user says stop

A reply-controlled loop uses fresh input to decide whether another round is wanted. Identify which actions belong to one round and where the next reply will be read. A reply read only before the loop cannot change while that loop runs.

Choose a loop structure based on whether the body must run at least once. If the condition follows the body, any reply variable used in that condition must still be in scope there. Decide which replies continue and how you will handle different letter cases.

For a password loop, compare the entered String with `.equals`, not `==`. For an input-validation loop, continue while the entry is invalid. In each case, identify what must change to make the condition false. Use the short counter-based `do-while` example to review the syntax before writing your own reply-controlled loop.

Sentinel, counter and validation loops

Pattern	What controls the loop?	Where it fits
Counter-controlled	A known number of repetitions.	Ten inputs, twenty dice rolls.
Sentinel-controlled	A special value that ends input.	Add numbers until zero or a negative number is entered.
Validation	An input is not yet acceptable.	Re-prompt for a number in a required range.
Repeat on request	The user’s reply.	Play again for Y or y.

Choose the condition from the question. If the sentinel is negative, any negative number stops the loop; testing only `number != -1` would implement a different rule. Keep a stop value out of the total when the question says it should not count.

The printf format specifiers

Every conversion specifier must match the type of its argument, and arguments follow the same order as the specifiers.

Specifier	Meaning	Example value
`%d`	Integer.	`12`
`%f`	Floating-point number; six decimal places by default.	`12.500000`
`%.1f`	One decimal place.	`12.5`
`%.2f`	Two decimal places.	`12.50`
`%s`	Text.	`Monday`
`%c`	One character.	`A`
`%b`	Boolean.	`true`
`%n`	A newline; no argument.	Moves to the next line.
`%%`	A percent sign; no argument.	`%`

For example, `System.out.printf("%s: %d items, %.2f each%n", "Pens", 3, 1.25);` displays `Pens: 3 items, 1.25 each`. `%d` cannot format a `double`; assigning the wrong type to a specifier causes a formatting exception.

Precision, grouping and field widths

```
double amount = 12345.6789;
System.out.printf("One decimal: %.1f%n", amount);
System.out.printf("Grouped: %,.2f%n", amount);
System.out.printf("%-12s %8.2f%n", "Distance:", 20.1125);
System.out.printf("Ticket: %04d%n", 37);
```

The first line displays 12345.7. With English/Ireland locale settings, the grouped value is 12,345.68. Grouping and decimal symbols depend on locale.

`%-12s` left-aligns text; `%8.2f` right-aligns a decimal in at least eight characters with two decimal places. `%04d` pads an integer with zeroes to at least four characters, so the last line displays `Ticket: 0037`. Width is a minimum; it does not chop off a longer value.

Formatted percentage and conversion reports

Calculate first, format second. A percentage expresses a part relative to a whole. This separate progress example uses fixed values.

```
double completed = 7.0;
double planned = 20.0;
double progress = completed / planned * 100.0;
System.out.printf("Progress: %.1f%%%n", progress);
```

This displays `Progress: 35.0%`. The `%%` adds a literal percent sign; it does not multiply the value by 100. If a user supplies a percentage to be used as a rate, convert it to a decimal first. For a conversion report, identify the original and resulting units, then choose labels and precision appropriate to each value.

Checking that a loop finishes

Before running a loop, identify its starting values, condition, body and update. Then trace one normal case and a boundary case.

Symptom	Likely issue
One too many or too few outputs	Check < versus <= and the starting count.
The same line appears forever	The update or next input is missing or cannot be reached.
The total includes the stop value	Add only after confirming it is not the sentinel.
A product stays at zero	Start the product at one.
A valid boundary is rejected	Check whether both range endpoints are meant to be included.

If a program loops endlessly in IntelliJ, stop that run before trying again. Make one correction and predict the next short run rather than repeatedly launching the same loop.

Check your understanding

1. How many times does `while (count < 3)` run if `count` begins at 3?
2. Where must an input loop read its next value?
3. Why is the sentinel excluded from the total?
4. Which placeholder would you use to display a `double` to two decimal places?

End of Week 5

CHAPTER 07

Week 6 · Types, text and chance

Casting, calculations, characters, strings and random numbers

Week 6 overview

Start with one small operation, run it, and change one value. Then combine the same ideas with the input, decisions and loops from earlier weeks. These notes support all three revised Week 6 labs.

The opening worked examples give complete programs with matching class names and filenames. Later sections explain the methods and planning decisions you will need, using small examples in different contexts. Use those ideas to develop your own lab programs. Fragments labelled **Inside `main`** belong inside a program's `main` method.

The three Week 6 labs

Lab	Start with	Build towards
Casting and Calculations	One decimal-to-integer cast, then one integer-to-double cast.	Averages, compound assignment, `Math`, constants and labelled calculations.
Characters, Strings and Repeated Input	Display one character.	Inspect, count and reverse text; classify characters; repeat calculations.
Random Numbers and Games	Generate one number from 0 to 9.	Inclusive ranges, guesses, quizzes, dice, coin flips and scorekeeping.

Keep the first programs small. There is no need to add keyboard input or formatted output until an exercise asks for it.

A first cast: decimal to integer

A **cast** converts a value to another type. This first example does only one conversion and prints one result. Save it as `Week6Casting1.java`; no import is needed.

```
public class Week6Casting1 {
    public static void main(String[] args) {
        double number = 12.75;
        int wholeNumber = (int) number;
        System.out.println(wholeNumber);
    }
}
```

CONSOLE OUTPUT

```
12
```

`(int) number` converts the decimal value to an integer. For values in the integer range, it discards the fractional part **towards zero**. It does not round: `12.75` becomes `12`. The original `number` variable still holds `12.75`.

Try one change: replace `12.75` with `8.9`. The result should be `8`.

A second cast: integer to double

Use the same pattern in the opposite direction. Save this complete program as `Week6Casting2.java`.

```
public class Week6Casting2 {
    public static void main(String[] args) {
        int number = 7;
        double decimalNumber = (double) number;
        System.out.println(decimalNumber);
    }
}
```

CONSOLE OUTPUT

```
7.0
```

The value is preserved: the whole number 7 becomes the double value 7.0. Java also allows `double decimalNumber = number;` without a cast. Writing `(double)` here makes the conversion explicit while you learn the pattern.

Try one change: replace `7` with `10`. The output should be `10.0`.

Widening, narrowing and rounding

Converting an `int` to a `double` is **widening** and can happen automatically. Every `int` value is represented exactly by `double`. Converting a `double` to an `int` is **narrowing** and needs an explicit cast because information may be lost.

Expression	Result	What happened
`(int) 7.9`	`7`	The fractional part was discarded.
`(int) -7.9`	`-7`	Truncation is towards zero, not down to -8.
`Math.round(7.9)`	`8`	Rounding is a different operation; this overload returns a `long`.
`(double) 7`	`7.0`	The integer value was preserved.

Use ordinary values within the target range for these exercises. A cast is not an input check and does not guarantee that every source value will fit the destination type.

Keeping the decimal part of an average

An average needs division after the values have been added. If both operands of `/` are integers, Java uses integer division.

Inside `main`:

```
int pages = 11;
System.out.println(pages / 4);
System.out.println((double) pages / 4);
```

The outputs are `2` and `2.75`. Both operands in the first division are integers. The cast in the second expression makes one operand a `double`, so the calculation preserves the fractional part.

The cast must happen **before** division. `(double) (pages / 4)` gives `2.0`: the fraction has already been discarded. The variable `pages` remains an `int`; the cast affects the value used in this expression.

For the lab average, identify the accumulated value and how many inputs it represents. Decide where the accumulation ends and where division belongs. A count of zero cannot be used as the divisor.

Compound assignment operators

A compound assignment updates a variable using its current value. In these integer examples, `total += value` has the same effect as `total = total + value`.

Statement	Starting value	New value
`value += 4;`	17	21
`value -= 4;`	17	13
`value *= 3;`	17	51
`value /= 4;`	17	4
`value %= 4;`	17	1

Each row starts again from 17. Applied in sequence, the results will be different. Integer `/=` discards the fractional part; `%=` stores the remainder. Trace the lab's sequence using its own starting value and operands.

Math.pow and compound growth

`Math` is available without an import. Call its methods using the class name: `Math.pow(base, exponent)` raises a value to a power and returns a `double`. Java's `^` operator does not mean exponentiation.

Inside `main`: one power, independent of a financial calculation.

```
double cube = Math.pow(2.0, 3);
System.out.println(cube);
```

CONSOLE OUTPUT

```
8.0
```

The first argument is the base; the second is the exponent. Reversing them generally changes the result. A power with exponent zero is 1 for the ordinary positive bases used here.

In the growth exercise, use the formula supplied in the lab. Label the starting amount, rate and time separately before translating it into Java. Convert a percentage to a decimal rate first: for example, 8% means `0.08`. Check that the time unit matches how often growth is applied. What should happen when no time has passed?

Math.sqrt and distance between points

The straight-line distance between two points comes from the square root of the sum of the squared coordinate differences.

Inside `main`:

```
double root = Math.sqrt(49.0);
System.out.println(root);
```

CONSOLE OUTPUT

```
7.0
```

`Math.sqrt` takes one argument and returns a `double`. A negative argument has no real square root; Java returns `NaN` (not a number) for it.

For the coordinate exercise, sketch the points and identify the horizontal and vertical differences. Use the lab's distance formula and give intermediate quantities meaningful names. Keep parentheses around grouped operations. Identical points should give zero distance, and exchanging the points should leave the distance unchanged. Negative coordinates are valid.

Math.PI, constants and circle measurements

`final` marks a variable that cannot be reassigned. A constant name should describe the value; uppercase with underscores is the usual convention. `Math.PI` already provides pi, so do not type a short approximation such as 3.14.

Inside `main`:

```
final double PI = Math.PI;
System.out.printf("Pi to four decimal places: %.4f%n", PI);
```

CONSOLE OUTPUT

```
Pi to four decimal places: 3.1416
```

The name `PI` can now be used in expressions, but assigning a new value to it would be a compile error. `Math.PI` can also be used directly.

For the circle task, distinguish the two requested measurements before implementing their formulas. Circumference uses the same length unit as the radius; area uses square units. Decide how to handle a negative radius. Formatting changes the displayed precision, not the stored result.

Tax calculations and aligned receipts

Keep a percentage, the amount it produces and a final total as distinct quantities. The lab supplies its exercise rate; check whether the entered price is before or after the added amount.

Inside `main`:

```
double minutes = 6.5;
System.out.printf("%-12s %8.2f%n", "Time:", minutes);
```

This unrelated timing example shows the formatting only. `%-12s` left-aligns the label in a field at least 12 characters wide; `%8.2f` right-aligns the number with two decimal places. Use consistent widths for an aligned report, and match each placeholder to the correct variable. In the lab, decide which intermediate values to calculate before displaying any receipt lines.

A monthly loan payment calculation

This task brings together types, units, a power and a decision. It assumes a fixed annual rate and equal monthly payments. Use the formula in the lab rather than trying to invent a financial formula.

Quantity	Meaning	Check before using it
Principal	Starting amount borrowed.	Must be positive.
Annual percentage	Rate as entered by the user.	Must not be negative; a percentage is not yet a decimal rate.
Monthly rate	Rate for one payment period.	Must use the same time unit as the payment schedule.
Payment count	Total number of monthly payments.	Must be positive and use a whole-number type.
Payment amount	Result for one month.	Keep as a decimal; format only for display.

Before coding, label every symbol in the supplied formula with one of your variable names. Work through the unit conversions on paper. Identify any repeated expression that would benefit from a named intermediate value.

Zero interest needs special attention because the general formula has a zero denominator in that case. Decide which decision structure can keep that invalid division from executing. Use the lab's check cases to assess your own implementation.

A first character program

A `char` stores one character value. Use single quotes for a character literal. Save this complete program as `Week6Text1.java`; no import is needed.

```
public class Week6Text1 {
    public static void main(String[] args) {
        char letter = 'a';
        System.out.println(letter);
    }
}
```

CONSOLE OUTPUT

```
a
```

Change `'a'` to `'b'` and run it again. This small example is the starting point for the character and text lab. A `String`, such as `"Java"`, uses double quotes and holds a sequence of characters.

Reading text and one character

For keyboard input, add `import java.util.Scanner;` above the class and create one `Scanner input = new Scanner(System.in);` inside `main`.

Inside `main`:

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a line: ");
String text = input.nextLine();
System.out.print("Enter one character: ");
char target = input.next().charAt(0);
```

`nextLine()` preserves spaces and can return an empty line. `next()` reads a non-empty token, and `charAt(0)` selects its first character; enter one letter or digit when the question asks for one character.

There is no `nextChar()` method. After `nextInt()` or `nextDouble()`, consume the leftover line ending before a later `nextLine()`; the Week 3 chapter shows that pattern.

String length and letter case

Inside `main`:

```
String label = "Desk";
System.out.println(label.length());
```

CONSOLE OUTPUT

```
4
```

Expression	Result	Purpose
`"ab cd".length()`	`5`	Count characters, including the space.
`"Quiet".toLowerCase()`	`"quiet"`	Return a lowercase version.
`"lamp".toUpperCase()`	`"LAMP"`	Return an uppercase version.

These methods do not change the original String. Assign a returned String to a variable if you want to keep it. `String` and `Character` need no additional import. For the lab, decide which results you need to store and which can be displayed directly.

Valid String indexes

Indexes start at zero. The third character is at index 2; the final valid index is `length() - 1`. Check length before accessing a position that may not exist.

Inside `main`:

```
String word = "Code";
System.out.println(word.charAt(1));
```

This prints `o`: index 1 is the second character.

Index	0	1	2	3
Character in `Code`	C	o	d	e

An empty string has length zero and has no valid character index. An invalid `charAt` call causes an exception; it does not return a blank. Before reading an optional character in user-entered text, ask whether that position exists and provide a useful alternative when it does not.

Counting a chosen character

Use one variable for the position being inspected and another for the number of matches. These variables describe different things.

Variable role	What it tells you	Question to ask
Position	Which character is being inspected?	Is this a valid String index?
Match count	How many inspected characters matched?	Should it change for a non-match?

A single comparison such as `text.charAt(position) == wanted` produces a boolean, provided the position is valid. It does not count anything by itself. The comparison is case-sensitive: `'r'` and `'R'` differ.

Plan how to visit every character once. Which update must happen even when the comparison is false? Trace a short word by hand before implementing the loop. Check a word with no matches as well as one with repeated matches.

Reversing a word

Reversing changes the order of characters. It is different from changing case or sorting letters. This exercise practises a loop that moves backwards through valid indexes.

Before coding, draw index boxes for a short word of your own. Mark the last character, then work towards the first. Use the String-index explanation to choose the starting position.

Planning questions:

- Which position holds the last character?
- Which direction must the index move?
- Which position must still be included before stopping?
- Where will the result being built be stored?

A one-letter word should stay the same. Be careful with an empty String: it has no character to read. Starting at `word.length()` would be one position too far.

Character comparisons and classification

Character comparisons use their numeric values, not language-specific dictionary order. For example, `'A'` has value 65 and `'a'` has value 97, so `'A' < 'a'` is true.

Inside `main`:

```
char symbol = '8';
System.out.println(Character.isDigit(symbol));
```

CONSOLE OUTPUT

```
true
```

Method	Result type	What it asks or produces
`Character.isLetter(value)`	`boolean`	Is the character a letter?
`Character.isDigit(value)`	`boolean`	Is it a digit?
`Character.toUpperCase(value)`	`char`	Its uppercase equivalent, when one exists.

`<`, `>`, `==` and `!=` all work with `char`. Use `==` for comparison and `=` for assignment. Choose the appropriate methods for the lab's requested analysis; do not confuse a boolean answer with a transformed character.

Character casts and Unicode limits

Casting an integer to `char` interprets it as a UTF-16 code unit. A Java `char` has values from 0 to 65535 inclusive. An out-of-range integer cast can wrap, so it is not a substitute for validation.

Inside `main`:

```
System.out.println((char) 67);
```

This prints `C`. Casting in the opposite direction, from `char` to `int`, reveals the numeric code unit. The digit character `'8'` is not the integer value 8.

For user input, decide how to report values outside the permitted range before attempting conversion. Not all values within the range are printable. Many emoji need two UTF-16 units, so one `char` is not enough for them. Use `Character.toUpperCase` for case conversion; arithmetic based on basic English letters is not a general Unicode rule.

Repeating a calculation with do-while

`do-while` performs the action before deciding whether to repeat. This small counter example shows the order of execution without adding a calculation or keyboard input.

Inside `main`:

```
int step = 2;
do {
    System.out.println(step);
    step--;
} while (step > 0);
```

This prints 2, then 1. The final `while (condition);` needs a semicolon. The counter-tested `while` pattern from Week 5 can run zero times; `do-while` always runs its body once before testing.

For the lab's repeated calculation, identify what belongs to one round and what decides whether another round starts. Read a fresh reply during each round so the decision can change. Keep the input variables in scope wherever they are needed, including the condition after the closing brace. Write and check the calculation once before adding repetition.

Building a text summary

A report presents results produced earlier. Develop and check the input and analysis separately before combining them with output formatting.

Inside `main`: this unrelated measurement shows two types of field.

```
String room = "Studio";
int seats = 18;
System.out.printf("%-12s %4d%n", room, seats);
```

`%s` accepts text and `%d` accepts an integer. The widths align fields; they do not perform the analysis. List the results your lab report needs and identify where each one will come from. Avoid storing a known sample answer in place of a calculation.

A first random-number program

Random numbers let a program make a choice for a game or simulation. This complete program generates one number and prints it. Save it as `Week6Random1.java`, including the import at the top.

```
import java.util.Random;

public class Week6Random1 {
    public static void main(String[] args) {
        Random random = new Random();
        int number = random.nextInt(10);
        System.out.println(number);
    }
}
```

`new Random()` creates the generator. `nextInt(10)` gives an integer from 0 up to, but excluding, 10: the possible values are **0 through 9**.

Run it several times. Results may change, and the same number can appear again. This program needs no keyboard input. A sample value is an example of one run, not an answer you must reproduce.

Inclusive random ranges

Create one generator inside `main` and reuse it: `Random random = new Random();`. Add `import java.util.Random;` above the class.

Expression	Minimum	Maximum
`random.nextInt(4)`	0	3
`random.nextInt(5) + 2`	2	6
`random.nextInt(5) + 10`	10	14

The general expression is `random.nextInt(max - min + 1) + min`. The `+1` includes both endpoints; the final `+min` shifts the range. The bound must be positive, so `max` must be at least `min`.

For 10–14, there are `14 - 10 + 1`, or 5, possible integers. Work through both endpoints before substituting the lab's own bounds. A few random runs need not happen to produce every possible endpoint.

One guess and one comparison

An equality comparison gives a boolean. Here is a small example about two fixed identifiers, separate from a guessing game.

Inside `main`:

```
int shelf = 4;
int requestedShelf = 3;
System.out.println(shelf == requestedShelf);
```

The result is `false`. Use `==` to compare integer values; `=` assigns a value.

For the lab, identify where the two compared values come from: one is generated, the other entered. Keep them in different variables. Decide which result your decision should display in each case. A comparison should not generate a replacement target. Keyboard input and random generation need the `Scanner` and `Random` imports respectively.

A quiz loop and a score

A quiz has a limit on how many questions are asked and a separate measure of success. Those are different responsibilities.

Responsibility	Question to answer in your plan
Question number	How will the program know when enough questions have been asked?
Question data	Which values must be fresh for each question?
Entered response	When does the program have enough information to judge it?
Score	Which event should change it?
Feedback	What should the learner see for each possible outcome?

Sketch one question before designing the loop. Then identify which quantities survive between questions and which are replaced. Trace a wrong response as carefully as a correct one: the question sequence must still make progress. Use the range explanation to derive the requested operand bounds yourself.

Dice rolls, totals and frequency counts

A total and a frequency count answer different questions. Consider these fixed observations of customers arriving at a desk:

Observation	Customers arriving
First	2
Second	4
Third	2

The total number of customers is 8. The frequency of an arrival group of size 2 is two observations. A frequency counts matching events; it does not add their numeric values.

For the simulation, list the statistics requested in the lab. Give each statistic its own variable and decide which event changes it. Derive the die's permitted range using the inclusive-range method. Plan a short trace to check that the number of trials, total and frequency have not been confused. Your random sequence will differ from another run.

A coin flip and a valid choice

A simulation can represent outcomes using numbers, then translate them into labels. Write down the outcomes you need and choose a consistent representation before coding. Check that the generator can produce every intended outcome and no others.

Validation is a separate decision: is the entered choice one of the permitted choices? Revisit Week 4's AND/OR rules when expressing that condition. Character case-conversion methods can help when uppercase and lowercase replies should be equivalent.

In your plan, distinguish an invalid entry, a valid round and a winning round. Should each one affect the statistics in the same way? Decide when the play-again question belongs. Trace one invalid reply followed by a valid reply to check the intended behaviour.

Two dice and a percentage

Each die needs its own generated value. Reusing one value for both would make them identical on every trial. Identify what counts as one successful trial using the rule in the lab.

This separate example calculates the percentage of flagged records in a fixed data set.

Inside `main`:

```
int flaggedRecords = 9;
int records = 40;
double percent = (double) flaggedRecords / records * 100.0;
System.out.printf("Flagged: %.1f%%%n", percent);
```

The output is `Flagged: 22.5%`. The cast prevents integer division; `%%` displays one percent sign. The denominator must be positive.

For your experiment, decide which count represents successful trials and which represents all trials. A random result is an observation, so it need not match another run or the theoretical probability exactly.

Guess until correct

This exercise combines a persistent target with changing input and feedback. The stopping rule comes from the relationship between the latest guess and the target.

Before implementing it, answer these questions:

- Which value must stay the same for the whole game?
- Which value must be read again on each attempt?
- Which loop structure allows at least one guess?
- Which comparisons distinguish the possible feedback messages?
- Where will the program store the total number of attempts?

Write a paper trace containing a high guess, a low guess and a correct guess. Check the lab's rule for which submissions count as attempts. Make sure the successful attempt is represented consistently. Revisit the earlier sections on comparisons, loop termination and random ranges as you develop your own program.

Lab 1: Casting and Calculations

Exercise	Relevant notes
1 · Decimal to integer	A first cast; widening, narrowing and rounding.
2 · Integer to double	A second cast.
3 · Total and average	Keeping the decimal part; compound assignment; Week 5 loops and `printf`.
4 · Compound interest	`Math.pow` and compound growth.
5 · Distance	`Math.sqrt` and distance between points.
6 · Circle	`Math.PI`, constants and circle measurements.
7 · Receipt	Tax calculations and aligned receipts.
8 · Loan	A monthly loan payment calculation.

Lab 2: Characters, Strings and Repeated Input

Exercise	Relevant notes
1 · Display a character	A first character program.
2 · Explore text	Reading text; String length and case; valid indexes.
3 · Count a character	Counting a chosen character.
4 · Reverse a word	Reversing a word.
5 · Analyse a character	Character comparisons and classification; character casts.
6 · Numeric value to character	Character casts and Unicode limits.
7 · Repeat averages	Repeating a calculation with `do-while`.
8 · Text report	Building a text summary; counting; Week 5 field widths.

Lab 3: Random Numbers and Games

Exercise	Relevant notes
1 · Generate a number	A first random-number program.
2 · Inclusive ranges	Inclusive random ranges.
3 · One guess	One guess and one comparison.
4 · Addition quiz	A quiz loop and a score.
5 · Dice statistics	Dice rolls, totals and frequency counts.
6 · Coin-flip game	A coin flip and a valid choice; `do-while`.
7 · Two-dice experiment	Two dice and a percentage.
8 · Guess until correct	Guess until correct.

Check your understanding

1. What does `(int) 6.8` produce? What about `(int) -6.8`?
2. Why does casting after integer division fail to recover the fraction?
3. What is the last valid index in a String of length 5?
4. Where must the index update go in a character-counting loop?
5. What are the minimum and maximum of `random.nextInt(5) + 10`?
6. Why are two generator calls needed for two dice?
7. Which variables change on every quiz question, and which change only on a correct answer?
8. Why does a zero-rate loan need a separate calculation?

End of Week 6

CHAPTER 08

Review · Weeks 1–6

A guide to revisiting the main ideas

Weeks 1–6 at a glance

Use this guide to choose what to revisit. A short example should make sense before you combine several ideas in a lab program.

Week	What to revisit
1 · Foundations	Source code, compilation, JVM, class and main, statements, braces, output and variables.
2 · Output and calculations	Types, escapes, arithmetic, precedence, integer division, remainders, percentages, constants, conversions and comments.
3 · Input and decisions	Scanner imports and methods, whole lines, numeric input, relational operators, if, else and nested decisions.
4 · More decisions	Else-if chains, logical operators, ranges, text comparison, switch cases, break and default.
5 · Repetition and output	While, do-while, counters, totals, products, sentinels, validation, repeat-on-request and printf.
6 · Types, text and chance	Casts, averages, Math formulas, constants, characters, String methods and loops, Random ranges, games and statistics.

For each topic, explain what the code does, predict an output, and choose one boundary input. Check both the result and whether the program finishes.